

Relatorio Tecnico - AV1

Disciplina: Inteligencia Artificial e Computacional (0700M8)

Professor: Daniel Leal Souza

Semestre: 01/2026

Equipe: [Preencher nomes]

1. Introducao e objetivo

Este trabalho foi desenvolvido em conformidade com a lauda da AV1, contemplando duas partes obrigatorias:

1. Parte 1: escolha de uma categoria de arvores de busca (foi escolhida a ****OPCA0 A - Busca Heuristica****).
2. Parte 2: ****Arvores de Decisao**** (obrigatoria para todas as equipes).

Objetivo geral: modelar problemas de inteligencia artificial, implementar algoritmos, realizar analise experimental, interpretar resultados e apresentar evidencias reproduziveis.

2. Ferramentas utilizadas e organizacao da equipe

- Linguagem: Python 3.13
- Bibliotecas: scikit-learn, numpy, matplotlib
- Ambiente: VS Code / terminal PowerShell
- Estrutura do projeto:
 - `src/heuristic\_search.py`
 - `src/decision\_tree\_part.py`
 - `src/main.py`
 - `outputs/` (evidencias)

Organizacao da equipe (preencher com os nomes reais):

- Integrante A: modelagem e implementacao da busca
- Integrante B: implementacao e analise da arvore de decisao
- Integrante C: validacao, resultados e visualizacoes
- Integrante D: consolidacao de relatorio e apresentacao

3. Parte 1 - OPCA0 A (Busca Heuristica)

3.0 BestFS como classe geral

Busca pela Melhor Escolha (BestFS) e a classe de algoritmos que seleciona o proximo no da fronteira pela melhor avaliacao de $f(n)$. Neste trabalho, foram instanciadas tres estrategias dentro desse paradigma:

- Busca Gulosa: $f(n)=h(n)$
- Busca Gulosa variante (Beam Greedy): expansao gulosa com largura de feixe $k=3$
- A* e Weighted A*: $f(n)=g(n)+w*h(n)$

3.1 Descricao formal do problema

Foi modelado um problema de roteamento em grade ponderada.

- Espaco de estados: celulas de um grid 14x10
- Estado inicial: $(0, 0)$
- Estado objetivo: $(13, 9)$
- Acoes validas: movimentos ortogonais (cima, baixo, esquerda, direita)
- Restricoes: celulas bloqueadas (obstaculos)
- Funcao de custo: custo de deslocamento para a celula de destino
 - custo padrao = 1
 - algumas celulas possuem custo maior (terreno dificil)

3.2 Algoritmos implementados

Foram implementados:

- Greedy Best-First Search (Busca Gulosa)
- Variante de Busca Gulosa: Greedy Beam Best-First com $k=3$

- A*
- Variante de A*: Weighted A* com $w=1.4$

Avaliacao:

- Greedy: $f(n)=h(n)$
- A*: $f(n)=g(n)+h(n)$
- Weighted A*: $f(n)=g(n)+w*h(n)$

3.3 Heurísticas utilizadas

Foram comparadas duas heurísticas para o mesmo problema:

- Distancia Manhattan
- Distancia Euclidiana

Estas heurísticas foram selecionadas por serem clássicas para espaços gradeados e permitirem comparação informatividade. Para este problema, ambas se mostraram admissíveis e consistentes nas checagens automatizadas.

Métodos de construção aplicados conforme a lauda:

- Método 1: distancia Manhattan
- Método 2: distancia Euclidiana

3.4 Admissibilidade e consistência

Para discutir propriedades teóricas, foi implementada verificação automática com custo real mínimo via Dijkstra reverso.

- Admissibilidade: verifica se $h(n) \leq h^*(n)$ para os nós válidos.
- Consistência: verifica se $h(n) \leq c(n, n') + h(n')$ para pares vizinhos.

Resultados destas checagens estão no arquivo:

- `outputs/heuristic_search/heuristic_validity_checks.json`

3.5 Evidências de execução e métricas

Para cada combinação algoritmo-heurística, foram registrados:

- caminho encontrado
- custo final
- número de nós expandidos
- tempo de execução
- memória de pico durante a execução
- amostras de valores $g(n)$, $h(n)$ e $f(n)$
- ordem de expansão

Arquivos:

- `outputs/heuristic_search/heuristic_metrics.csv`
- `outputs/heuristic_search/heuristic_paths_and_expansions.json`
- `outputs/heuristic_search/*.png`

3.6 Resultados obtidos

Os resultados executados estão sintetizados abaixo.

Algoritmo	Heurística	Custo final	Nos expandidos	Tempo (ms)	Memoria pico (KB)
---	---	---	---	---	---
Greedy Best-First Search	Manhattan	22	23	0.368	7.15
Greedy Beam Best-First (k=3)	Manhattan	22	57	0.878	13.23
A*	Manhattan	22	67	0.580	25.53
Weighted A*	Manhattan	22	23	0.202	11.74
Greedy Best-First Search	Euclidiana	31	23	0.196	14.09
Greedy Beam Best-First (k=3)	Euclidiana	22	44	0.456	17.43
A*	Euclidiana	22	73	0.584	26.80

| Weighted A* | Euclidiana | 22 | 38 | 0.517 | 18.05 |

Conclusões principais:

- A heurística Manhattan foi mais eficiente neste grid, mantendo admissibilidade e consistência e produzindo a melhor orientação para a meta.
- Greedy com Euclidiana foi mais rápido, mas encontrou solução pior em custo, mostrando a fragilidade de algoritmos gulos apenas $h(n)$.
- A variante gulosa em feixe ($k=3$) com Euclidiana corrigiu o custo da busca gulosa simples (de 31 para 22), mostrando ganho de qualidade por maior largura de exploração.
- A* preservou o melhor custo encontrado entre os experimentos, a custo de expandir mais nós.
- Weighted A* reduziu expansões em relação ao A* em alguns cenários, oferecendo um meio-termo prático.

4. Parte 2 - Árvores de Decisão

4.1 Dataset e pré-processamento

Foi utilizado o dataset Breast Cancer Wisconsin (Diagnostic) da biblioteca scikit-learn.

- Tarefa: classificação binária
- Instâncias: 569
- Atributos: 30
- Variável-alvo: classe diagnóstica (maligno/benigno)

Não houve necessidade de imputação de nulos neste dataset. O conjunto foi dividido em treino e teste com estratificação para preservar a proporção das classes.

4.2 Critérios de divisão e construção

Foram comparados critérios:

- Gini
- Entropia (ganho de informação)

Conceitos aplicados:

- escolha do atributo da raiz por maior redução de impureza
- partições recursivas
- condições de parada por pureza/profundidade
- relação entre pureza, entropia e ganho de informação

4.3 Validação, teste e poda

Procedimento experimental:

- split treino/teste (80/20) estratificado
- validação cruzada estratificada com 5 folds no treino

Configurações comparadas:

1. `gini\_unpruned`
2. `entropy\_unpruned`
3. `gini\_depth4`
4. `entropy\_depth6\_pruned` (com `ccp\_alpha`)

Arquivos de saída:

- `outputs/decision\_tree/decision\_tree\_metrics.csv`
- `outputs/decision\_tree/decision\_tree\_summary.json`
- `outputs/decision\_tree/decision\_tree\_classification\_report.txt`
- `outputs/decision\_tree/decision\_tree\_rules.txt`
- `outputs/decision\_tree/decision\_tree\_visualization.png`

4.4 Resultados obtidos

Os resultados experimentais foram:

| Configuração | Critério | Profundidade | Poda | CV média | Teste acc. | F1 ponderado | Nós | Profundidade

```

real |
| --- | --- | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
| gini_unpruned | Gini | livre | nao | 0.9165 | 0.9123 | 0.9130 | 37 | 7 |
| entropy_unpruned | Entropia | livre | nao | 0.9297 | 0.9123 | 0.9133 | 31 | 6 |
| gini_depth4 | Gini | 4 | nao | 0.9297 | 0.9386 | 0.9387 | 21 | 4 |
| entropy_depth6_pruned | Entropia | 6 | sim | 0.9297 | 0.9123 | 0.9133 | 31 | 6 |

```

Leitura tecnica:

- A configuracao `gini\_depth4` foi a mais equilibrada neste experimento, combinando melhor acuracia de com arvore menor.
- Modelos sem restricao de profundidade ficaram mais complexos, com maior risco de sobreajuste.
- A poda e a limitacao de profundidade aumentam interpretabilidade e controlam a variancia.

4.5 Overfitting e interpretabilidade

Pontos discutidos:

- Modelos sem poda tendem a maior profundidade e maior risco de overfitting.
- Restricao de profundidade e poda por complexidade reduzem variancia e melhoram generalizacao.
- A visualizacao da arvore permite interpretacao de atributos mais relevantes proximos da raiz.

Interpretacao objetiva do modelo (a partir das regras extraidas):

- Atributo mais proximo da raiz: `worst radius` com limiar em `16.80`.
- Segundo nivel relevante: `worst concave points` com limiar em `0.14`.
- Outros atributos recorrentes nas primeiras decisoes: `area error`, `worst texture`, `worst concavity`, `mean texture`.
- Exemplo de regra interpretavel: quando `worst radius <= 16.80` e `worst concave points <= 0.14`, ha predominancia de classificacao benigna.

Tratamento de atributos continuos e limiares:

- O algoritmo CART do scikit-learn testa pontos de corte numericos (`feature <= threshold`) ao longo dos atributos continuos.
- Esses limiares sao aprendidos no treino para maximizar reducao de impureza (Gini/Entropia), como nos `16.80`, `0.14` e `25.62` observados nas regras exportadas.

5. Comparacao global e discussao integrada

As duas partes se complementam:

- Busca heuristica foca em planejamento orientado por funcao de avaliacao.
- Arvore de decisao foca em aprendizagem supervisionada interpretavel.

Em ambos os casos, desempenho depende de escolhas de modelagem:

- Busca: qualidade da heuristica e estrutura de custo.
- Arvore: criterio de divisao, profundidade, poda e validacao.

6. Limitacoes, dificuldades e melhorias

Limitacoes deste estudo:

- Um unico cenario principal de busca foi usado para analise.
- Arvore de decisao foi avaliada em um dataset padrao de referencia.

Melhorias futuras:

- incluir mais mapas e diferentes distribuicoes de custos
- testar novas variantes de busca informada (ex.: IDA*)
- comparar arvores com outros classificadores interpretableis
- ampliar analise estatistica com repeticoes e intervalos de confianca

7. Conclusao

O trabalho atendeu aos requisitos tecnicos da lauda para OPCA0 A e para a parte obrigatoria de Arvores e Decisao, apresentando modelagem, implementacao, validacao experimental, evidencias de execucao e interpretacao dos resultados.

A atividade consolidou conceitos centrais de IA relacionados a busca informada e aprendizagem de modelos interpretableis.

8. Referencias

- RUSSELL, S.; NORVIG, P. Artificial Intelligence: A Modern Approach.
- MITCHELL, T. Machine Learning.
- Documentacao oficial scikit-learn: <https://scikit-learn.org/>

9. Resultados resumidos para apresentacao

- Busca heuristica com Manhattan foi a configuracao mais consistente para a rota.
- A* encontrou o melhor custo final nos experimentos.
- A arvore `gini\_depth4` obteve melhor equilibrio entre desempenho e complexidade.
- O trabalho gerou evidencias objetivas em CSV, JSON, TXT e PNG.

10. Anexos

- Anexo A: evidencias em `outputs/`
- Anexo B: uso de IA em `docs/anexo\_uso\_ia.md`